byte values in that range. Other ASCII characters are allowed in `string`, and are ignored in calculating the check digit.

```
    char c;
    Int j,k=0,m=0,n=str.length();
    static Int ip[10][8]={{0,1,5,8,9,4,2,7},{1,5,8,9,4,2,7,0},
        {2,7,0,1,5,8,9,4},{3,6,3,6,3,6,3,6},{4,2,7,0,1,5,8,9},
        {5,8,9,4,2,7,0,1},{6,3,6,3,6,3,6,3},{7,0,1,5,8,9,4,2},
        {8,9,4,2,7,0,1,5},{9,4,2,7,0,1,5,8}};
    static Int ij[10][10]={{0,1,2,3,4,5,6,7,8,9},{1,2,3,4,0,6,7,8,9,5},
        {2,3,4,0,1,7,8,9,5,6},{3,4,0,1,2,8,9,5,6,7},{4,0,1,2,3,9,5,6,7,8},
        {5,9,8,7,6,0,4,3,2,1},{6,5,9,8,7,1,0,4,3,2},{7,6,5,9,8,2,1,0,4,3},
        {8,7,6,5,9,3,2,1,0,4},{9,8,7,6,5,4,3,2,1,0}};
        Group multiplication and permutation tables.
    for (j=0;j<n;j++) {                  Look at successive characters.
        c=str[j];
        if (c >= 48 && c <= 57)          Ignore everything except digits.
            k=ij[k][ip[(c+2) % 10][7 & m++]];
    }
    for (j=0;j<10;j++)                   Find which appended digit will check properly.
        if (ij[k][ip[j][m & 7]] == 0) break;
    ch=char(j+48);                       Convert to ASCII.
    return k==0;
}
```

CITED REFERENCES AND FURTHER READING:

Saadawi, T.N., and Ammar, M.H. 1994, *Fundamentals of Telecommunication Networks* (New York: Wiley).[1]

LeVan, J. 1987, "A Fast CRC," *Byte*, vol. 12, pp. 339–341 (November).[2]

Koopman, P., and Chakravarty, T. 2004, "Cyclic Redundancy Code (CRC) Polynomial Selection for Embedded Networks," in *International Conference on Dependable Systems and Networks (DSN-2004)* (IEEE Computer Society).[3]

Sarwate, D.V. 1988, "Computation of Cyclic Redundancy Checks via Table Look-Up," *Communications of the ACM*, vol. 31, pp. 1008–1013.[4]

Griffiths, G., and Stones, G.C. 1987, "The Tea-Leaf Reader Algorithm: An Efficient Implementation of CRC-16 and CRC-32," *Communications of the ACM*, vol. 30, pp. 617–620.[5]

Wagner, N.R., and Putter, P.S. 1989, "Error Detecting Decimal Digits," *Communications of the ACM*, vol. 32, pp. 106–110.[6]

22.5 Huffman Coding and Compression of Data

A lossless data compression algorithm takes a string of symbols (typically ASCII characters or bytes) and translates it *reversibly* into another string, one that is *on the average* of shorter length. The words "on the average" are crucial; it is obvious that no reversible algorithm can make all strings shorter — there just aren't enough short strings to be in one-to-one correspondence with longer strings. Compression algorithms are possible only when, on the input side, some strings, or some input symbols, are more common than others. These can then be encoded in fewer bits than rarer input strings or symbols, giving a net average gain. We already quantified this idea, with the concept of *entropy*, in §14.7.

There exist many, quite different, compression techniques, corresponding to different ways of detecting and using departures from equiprobability in input strings.

In this section and the next we shall consider only *variable length codes* with *defined word* inputs. In these, the input is sliced into fixed units, for example ASCII characters, while the corresponding output comes in chunks of variable size. The simplest such method is Huffman coding [1], discussed in this section. Another example, *arithmetic compression*, is discussed in §22.6.

At the opposite extreme from defined-word, variable length codes are schemes that divide up the *input* into units of variable length (words or phrases of English text, for example) and then transmit these, often with a fixed length output code. The most widely used code of this general type is the Ziv-Lempel code [2]. References [3-5] give the flavor of some other compression techniques, with references to the large literature.

The idea behind Huffman coding is simply to use shorter bit patterns for more common characters. Suppose the input alphabet has $N_{\rm ch}$ characters, and that these occur in the input string with respective probabilities p_i, $i = 1, \ldots, N_{\rm ch}$, so that $\sum p_i = 1$. As we saw in §14.7, strings consisting of independently random sequences of these characters (a conservative, but not always realistic assumption) require, on the average, at least

$$H = -\sum p_i \log_2 p_i \tag{22.5.1}$$

bits per character, where H is the entropy of the probability distribution. Moreover, coding schemes exist that approach the bound arbitrarily closely. For the case of equiprobable characters, with all $p_i = 1/N_{\rm ch}$, one easily sees that $H = \log_2 N_{\rm ch}$, which is the case of no compression at all. Any other set of p_i's gives a smaller entropy, allowing some useful compression.

Notice that the bound of (22.5.1) would be achieved if we could encode character i with a code of length $L_i = -\log_2 p_i$ bits: Equation (22.5.1) would then be the average $\sum p_i L_i$. The trouble with such a scheme is that $-\log_2 p_i$ is not generally an integer. How can we encode the letter "Q" in 5.32 bits? Huffman coding makes a stab at this by, in effect, approximating all the probabilities p_i by integer powers of 1/2, so that all the L_i's are integral. If all the p_i's are in fact of this form, then a Huffman code does achieve the entropy bound H.

The construction of a Huffman code is best illustrated by example. Imagine a language, Vowellish, with the $N_{\rm ch} = 5$ character alphabet A, E, I, O, and U, occurring with the respective probabilities 0.12, 0.42, 0.09, 0.30, and 0.07. Then the construction of a Huffman code for Vowellish is accomplished in the table on the next page.

Here is how it works, proceeding in sequence through $N_{\rm ch}$ stages, represented by the columns of the table. The first stage starts with $N_{\rm ch}$ nodes, one for each letter of the alphabet, containing their respective relative frequencies. At each stage, the two smallest probabilities are found, summed to make a new node, and then dropped from the list of active nodes. (A "block" denotes the stage where a node is dropped.) All active nodes (including the new composite) are then carried over to the next stage (column). In the table, the names assigned to new nodes (e.g., AUI) are inconsequential. In the example shown, it happens that (after stage 1) the two smallest nodes are always an original node and a composite one; this need not be true in general: The two smallest probabilities might be both original nodes, or both composites, or one of each. At the last stage, all nodes will have been collected into one grand composite of total probability 1.

Node	Stage:	1	2	3	4	5
1	A:	0.12	0.12 ■			
2	E:	0.42	0.42	0.42	0.42 ■	
3	I:	0.09 ■				
4	O:	0.30	0.30	0.30 ■		
5	U:	0.07 ■				
6		UI:	0.16 ■			
7			AUI:	0.28 ■		
8				AUIO:	0.58 ■	
9					EAUIO:	1.00

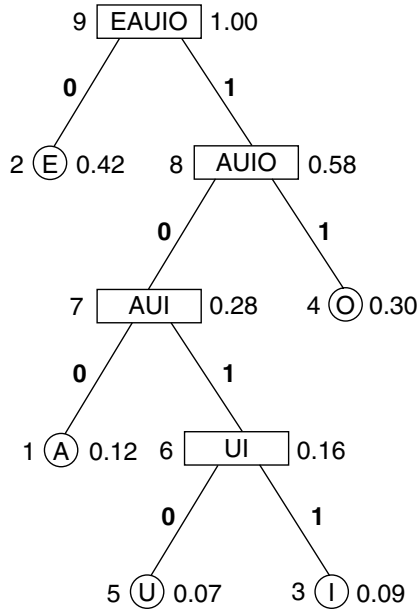

Figure 22.5.1. Huffman code for the fictitious language Vowellish, in tree form. A letter (A, E, I, O, or U) is encoded or decoded by traversing the tree from the top down; the code is the sequence of 0's and 1's on the branches. The value to the right of each node is its probability; to the left, its node number in the table.

Now, to see the code, you redraw the data in the table as a tree (Figure 22.5.1). As shown, each node of the tree corresponds to a node (row) in the table, indicated by the integer to its left and probability value to its right. Terminal nodes, so called, are shown as circles; these are single alphabetic characters. The branches of the tree are labeled 0 and 1. The code for a character is the sequence of zeros and ones that lead to it, from the top down. For example, E is simply 0, while U is 1010.

Any string of zeros and ones can now be decoded into an alphabetic sequence. Consider, for example, the string 1011111010. Starting at the top of the tree we

descend through 1011 to I, the first character. Since we have reached a terminal node, we reset to the top of the tree, next descending through 11 to O. Finally 1010 gives U. The string thus decodes to IOU.

These ideas are embodied in the following `Huffcode` object. The constructor lets you specify $N_{\rm ch}$, and an integer frequency-of-occurrence table of length $N_{\rm ch}$ telling how often each character occurs in some large corpus of text. These integers are, of course, proportional to the p_i's. The reason for using integers is so that any two computers will produce exactly the same code from the same input data. This might not be true if we used floating-point values. The constructor utilizes a heap structure (see §8.3) for efficiency; for a detailed description, see Sedgewick [6].

Once you have created an instance of `Huffcode`, you code a message by calling `codeone` for each message character in turn. This writes bits into a byte array `code` that you supply as an argument. There is no message-dependent saved state, so you could interleave different messages if there were some reason to do so.

Decoding a Huffman-encoded message is slightly more complicated. The coding tree must be traversed from the top down, using up a variable number of bits. This is done by the method `decodeone`.

There is no such thing as an "end of message" (EOM) marker in Huffman codes — not unless you provide one. Successive calls to `decodeone` will happily decode bits into characters until your hardware traps an illegal memory read! That is because every path on the tree (cf. Figure 22.5.1) terminates in a valid character. In practice, one increases $N_{\rm ch}$ by 1, and gives the extra character a frequency of occurrence of 1 (versus large values for the other characters). The new character becomes the EOM marker. Similarly, one can add other extra characters for other "out-of-band" signaling. If these occur rarely, the overhead on the message is negligible.

huffcode.h

```
struct Huffcode {
```

Object for Huffman encoding and decoding.

```
    Int nch,nodemax,mq;
    Int ilong,nlong;
    VecInt ncod,left,right;
    VecUint icod;
    Uint setbit[32];

    Huffcode(const Int nnch, VecInt_I &nfreq)
    : nch(nnch), mq(2*nch-1), icod(mq), ncod(mq), left(mq), right(mq) {
```

Constructor. Given the frequency of occurrence table `nfreq[0..nnch-1]` for `nnch` characters, constructs the Huffman code. Also sets `ilong` and `nlong` as the character number that produced the longest code symbol, and the length of that symbol.

```
        Int ibit,j,node,k,n,nused;
        VecInt index(mq), nprob(mq), up(mq);
        for (j=0;j<32;j++) setbit[j] = 1 << j;
        for (nused=0,j=0;j<nch;j++) {
            nprob[j]=nfreq[j];
            icod[j]=ncod[j]=0;
            if (nfreq[j] != 0) index[nused++]=j;
        }
        for (j=nused-1;j>=0;j--)
            heep(index,nprob,nused,j);
        k=nch;
        while (nused > 1) {
            node=index[0];
            index[0]=index[(nused--)-1];
            heep(index,nprob,nused,0);
            nprob[k]=nprob[index[0]]+nprob[node];
```

Sort `nprob` into a heap structure in `index`.

Combine heap nodes, remaking the heap at each stage.

```
            left[k]=node;                    Store left and right children of a node.
            right[k++]=index[0];
            up[index[0]] = -Int(k);          Indicate whether a node is a left or right child
            index[0]=k-1;                        of its parent.
            up[node]=k;
            heep(index,nprob,nused,0);
        }
        up[(nodemax=k)-1]=0;
        for (j=0;j<nch;j++) {                Make the Huffman code from the tree.
            if (nprob[j] != 0) {
                for (n=0,ibit=0,node=up[j];node;node=up[node-1],ibit++) {
                    if (node < 0) {
                        n |= setbit[ibit];
                        node = -node;
                    }
                }
                icod[j]=n;
                ncod[j]=ibit;
            }
        }
        nlong=0;
        for (j=0;j<nch;j++) {
            if (ncod[j] > nlong) {
                nlong=ncod[j];
                ilong=j;
            }
        }
        if (nlong > numeric_limits<Uint>::digits)
            throw("Code too long in Huffcode.  See text.");
    }
```

```
    void codeone(const Int ich, char *code, Int &nb) {
```

Huffman encode the single character `ich` (in the range `0..nch-1`), write the result to the byte array `code` starting at bit `nb` (whose smallest valid value is zero), and increment `nb` to the first unused bit. This routine is called repeatedly to encode consecutive characters in a message. The user is responsible for monitoring that the value of `nb` does not overrun the length of `code`.

```
        Int m,n,nc;
        if (ich >= nch) throw("bad ich (out of range) in Huffcode");
        if (ncod[ich]==0) throw("bad ich (zero prob) in Huffcode");
        for (n=ncod[ich]-1;n >= 0;n--,++nb) {        Loop over the bits in the stored
            nc=nb >> 3;                                  Huffman code for ich.
            m=nb & 7;
            if (m == 0) code[nc]=0;                  Set appropriate bits in code.
            if ((icod[ich] & setbit[n]) != 0) code[nc] |= setbit[m];
        }
    }
```

```
    Int decodeone(char *code, Int &nb) {
```

Starting at bit number `nb` in the byte array `code`, decode a single character (returned as `ich` in the range `0..nch-1`) and increment `nb` appropriately. Repeated calls, starting with `nb` = 0, will return successive characters in a compressed message. The user is responsible for detecting EOM from the message content.

```
        Int nc;
        Int node=nodemax-1;
        for (;;) {                               Set node to the top of the decoding tree, and
            nc=nb >> 3;                              loop until a valid character is obtained.
            node=((code[nc] & setbit[7 & nb++]) != 0 ?
                right[node] : left[node]);
                Branch left or right in tree, depending on its value.
            if (node < nch) return node;         If we reach a terminal node, we have a com-
        }                                            plete character and can return.
    }
```

```
    void heep(VecInt_IO &index, VecInt_IO &nprob, const Int n, const Int m) {
    Used by the constructor to maintain a heap structure in the array index[0..m-1].
        Int i=m,j,k;
        k=index[i];
        while (i < (n >> 1)) {
            if ((j = 2*i+1) < n-1
                && nprob[index[j]] > nprob[index[j+1]]) j++;
            if (nprob[k] <= nprob[index[j]]) break;
            index[i]=index[j];
            i=j;
        }
        index[i]=k;
    }
};
```

`Huffcode` requires that the longest code for a single character fits into your machine's integer wordlength (typically 32 bits), and will tell you if this is violated. If this happens, you'll need to increase the frequency-of-occurrence value for the rarest characters. This will affect your compression hardly at all.

It is a feature, not a bug, that `Huffcode` allows you to specify some characters as having zero frequency of occurrence, and then completely omits these from the code. This can be very useful when, for example, you want to compress a file consisting only of ASCII characters 0–9, +, −, and ".", as might occur in a file of numerical values. But don't then try to encode one of the omitted characters!

22.5.1 Run-Length Encoding

For the compression of highly correlated bit streams (for example the black or white values along a facsimile scan line), Huffman compression is often combined with *run-length encoding*: Instead of sending each bit, the input stream is converted to a series of integers indicating how many consecutive bits have the same value. These integers are then Huffman-compressed. The Group 3 CCITT facsimile standard functions in this manner, with a fixed, immutable, Huffman code, optimized for a set of eight standard documents [7].

CITED REFERENCES AND FURTHER READING:

Hamming, R.W. 1980, *Coding and Information Theory* (Englewood Cliffs, NJ: Prentice-Hall).

Huffman, D.A. 1952, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the Institute of Radio Engineers*, vol. 40, pp. 1098–1101.[1]

Ziv, J., and Lempel, A. 1978, "Compression of Individual Sequences via Variable-Rate Coding," *IEEE Transactions on Information Theory*, vol. IT-24, pp. 530–536.[2]

Sayood, K. 2005, *Introduction to Data Compression*, 3rd ed. (San Francisco: Morgan Kaufmann).[3]

Salomon, D. 2004, *Data Compression: The Complete Reference*, 3rd ed. (New York: Springer).[4]

Wayner, P. 1999, *Compression Algorithms for Real Programmers* (San Francisco: Morgan Kaufmann).[5]

Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapter 22.[6]

Hunter, R., and Robinson, A.H. 1980, "International Digital Facsimile Coding Standard," *Proceedings of the IEEE*, vol. 68, pp. 854–867.[7]